

Server-Side Request Forgery (SSRF)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un SSRF ocurre cuando la app acepta una URL (para descargar un avatar, verificar un webhook, importar un fichero remoto...) y el SERVIDOR la solicita el mismo. Si controlas esa URL, el servidor se convierte en tu proxy: puedes hacerle pedir localhost, la red interna, o metadatos de la nube, cosas a las que tu no llegas directamente.

El programador confía en que la URL apuntara a un recurso externo legítimo (una imagen, un webhook real) y no valida ni restringe el destino. El servidor, al estar dentro de la red interna, ve cosas que tu Kali no ve desde fuera.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando un parametro pide una URL, un dominio, o algo que 'importa' o 'verifica' un recurso remoto: url, webhook, avatar, import, callback, feed. Muy comun tambien en generadores de PDF que renderizan una URL.

Cómo identificarla

- Un parametro llamado url, webhook, avatar, import, feed o callback.
- La app describe una funcion de 'importar desde URL' o 'verificar disponibilidad'.
- Tu listener recibe la peticion cuando apuntas el parametro a tu IP.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Apunta el parametro a tu propio listener (python3 -m http.server 8000) antes de probar localhost o la red interna.

Evidencia esperada: Una peticion entrante en tu listener HTTP. Eso confirma que el servidor SI solicita URLs que tu controlas.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
python3 -m http.server 8000
```

Objetivo y contexto: Levanta un listener propio para confirmar el SSRF sin ambigüedad: si te llega la petición, sabes con certeza que el servidor sigue URLs que tu controlas.

Resultado esperado: El listener queda a la escucha. Combinalo con el siguiente comando para disparar la petición desde el servidor.

Capa 2: prueba documentada

```
curl -sS "$URL/fetch?url=http://ATTACKER_IP:8000/"
```

Objetivo y contexto: Dispara el SSRF apuntando al tu propio listener. Es la prueba mínima y segura antes de tocar localhost o redes internas.

Resultado esperado: Una petición GET registrada en tu python3 -m http.server. Confirmado esto, repite el mismo parámetro apuntando a 127.0.0.1 o al rango interno.

Capa 3: prueba documentada

```
http://127.1/
```

Objetivo y contexto: Bypass de filtros que bloquean la cadena '127.0.0.1' pero no normalizan la IP: 127.1 se expande al mismo loopback 127.0.0.1. Variantes: 127.0.1, 2130706433 (decimal), 0x7f000001 (hex), [::1] (IPv6).

Resultado esperado: Si con 127.1 pasa lo que con 127.0.0.1 estaba bloqueado, el filtro era por cadena; ya tienes acceso al loopback interno.

Anatomía de los payloads

La primera prueba es python3 -m http.server 8000. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una petición entrante en tu listener HTTP. Eso confirma que el servidor SI solicita URLs que tu controlas.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Confirma SIEMPRE contra tu propio listener primero, nunca contra localhost/metadata directamente.
- Una vez confirmado, prueba destinos internos: 127.0.0.1 con distintos puertos, rangos internos típicos, o el endpoint de metadatos de la nube (169.254.169.254) si el contexto es cloud.

- Si bloquea '127.0.0.1' o 'localhost' literalmente, es un filtro por cadena, no por rango: prueba representaciones equivalentes del loopback (127.1, 127.0.1, decimal 2130706433, 0x7f000001, [::1]). El fallo es validar el texto en vez de normalizar la IP y comprobar el rango.
- Si la salida muestra la respuesta cruda de una herramienta (un medidor de curl, cabeceras), el backend envuelve curl/wget: mira si puedes colar un esquema local (file://) o una segunda URL -> ver argument-injection.
- Si la respuesta refleja el contenido obtenido (SSRF 'visible'), lees directamente lo que hay en el destino interno.
- Si no refleja nada (SSRF 'ciega'), solo sabes que respondió o no; usa temporización o un servicio out-of-band para confirmar.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- La app valida el destino contra una lista blanca de dominios permitidos, sin excepciones.
- La 'URL' en realidad solo se usa para mostrar un enlace en el cliente, nunca la solicita el servidor.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.

2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Consigues que el SERVIDOR haga una petición HTTP a la URL que tu eliges, alcanzando redes internas que tu no ves directamente. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(ssrf\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.